

TÍNH TRỪU TƯỢNG (ABSTRACTION) TRONG JAVA

1. Khái niệm và Ý nghĩa

Trong đời sống thực tế, khi bạn sử dụng một chiếc xe hơi, bạn chỉ cần quan tâm đến việc sử dụng vô lăng, chân ga và chân phanh để điều khiển xe. Bạn không cần thiết phải hiểu rõ động cơ đốt trong hoạt động như thế nào, hệ thống phun xăng điện tử xử lý ra sao hay trục cam quay như thế nào. Những chi tiết phức tạp đó đã được "ẩn" đi bên dưới nắp ca-pô.

Trong Lập trình hướng đối tượng (OOP), **Tính Trừu tượng (Abstraction)** là cơ chế cho phép ẩn đi các chi tiết triển khai phức tạp bên trong và chỉ hiển thị ra bên ngoài những tính năng (phương thức) cần thiết cho người sử dụng.

Mục đích chính:

- **Giảm độ phức tạp:** Giúp lập trình viên tập trung vào việc đối tượng "làm gì" (What) thay vì "làm như thế nào" (How).
- **Tăng tính bảo mật:** Che giấu các dữ liệu hoặc logic xử lý nhạy cảm.
- **Dễ bảo trì và thay đổi:** Việc thay đổi code xử lý bên trong không ảnh hưởng đến các thành phần đang sử dụng nó bên ngoài.

Trong Java, tính trừu tượng được thực hiện thông qua hai công cụ chính:

1. **Lớp trừu tượng (Abstract Class)**
2. **Giao diện (Interface)**

2. Lớp Trừu tượng (Abstract Class)

2.1. Định nghĩa

Lớp trừu tượng là một lớp được khai báo với từ khóa **abstract**. Nó đóng vai trò là lớp cha cho các lớp khác kế thừa, nhưng bản thân nó không thể được sử dụng để tạo ra đối tượng (không thể dùng **new**).

2.2. Đặc điểm kỹ thuật

- **Phương thức trừu tượng:** Lớp trừu tượng có thể chứa các phương thức trừu tượng (abstract methods) - là các phương thức chỉ có khai báo tên, kiểu trả về và tham số nhưng **không có phần thân** (không có code xử lý).
- **Phương thức thường:** Nó cũng có thể chứa các phương thức thông thường (có phần thân) để cung cấp các logic chung cho các lớp con.
- **Kế thừa:** Khi một lớp con kế thừa lớp trừu tượng, nó **bắt buộc** phải ghi đè (override) và triển khai nội dung cho tất cả các phương thức trừu tượng của lớp cha (trừ khi lớp con cũng là abstract).

2.3. Ví dụ minh họa

```

// Khai báo lớp trừu tượng
abstract class DongVat {
    // Phương thức trừu tượng (không có thân hàm)
    abstract void tiengKeu();

    // Phương thức thường (có thân hàm)
    void ngu() {
        System.out.println("Zzz...");
    }
}

// Lớp con kế thừa
class Cho extends DongVat {
    // Bắt buộc phải triển khai phương thức trừu tượng
    @Override
    void tiengKeu() {
        System.out.println("Gâu gâu");
    }
}

class Meo extends DongVat {
    @Override
    void tiengKeu() {
        System.out.println("Meo meo");
    }
}

```

3. Giao diện (Interface)

3.1. Định nghĩa

Interface (Giao diện) là một bản thiết kế (blueprint) hoặc một "hợp đồng" (contract) quy định các hành vi mà một lớp phải tuân thủ. Trong Java, Interface được khai báo bằng từ khóa **interface**.

3.2. Đặc điểm kỹ thuật

- **Trừu tượng hoàn toàn:** Trước Java 8, Interface chỉ được chứa các phương thức trừu tượng và các hằng số (**public static final**).
- **Đa kế thừa (giả lập):** Một lớp có thể thực thi (**implements**) nhiều Interface cùng lúc. Đây là cách Java hỗ trợ đa kế thừa.
- **Không chứa trạng thái:** Interface không thể chứa các biến instance (thuộc tính lưu trữ trạng thái), chỉ chứa hằng số.

3.3. Ví dụ minh họa

```
// Khai báo Interface
interface DiChuyen {
    void chay(); // Mặc định là public abstract
}

interface AnUong {
    void an();
}

// Lớp Cho thực thi cùng lúc 2 Interface
class Cho implements DiChuyen, AnUong {
    @Override
    public void chay() {
        System.out.println("Chó chạy bằng 4 chân.");
    }

    @Override
    public void an() {
        System.out.println("Chó ăn xương.");
    }
}
```

4. So sánh Abstract Class và Interface

Đây là nội dung quan trọng để phân biệt khi nào dùng loại nào.

Tiêu chí	Lớp trừu tượng (Abstract Class)	Giao diện (Interface)
Từ khóa khai báo	abstract class	interface
Từ khóa kế thừa	extends	implements
Số lượng kế thừa	Một lớp chỉ kế thừa được một Abstract Class.	Một lớp có thể thực thi nhiều Interface.
Phương thức	Có thể chứa cả phương thức trừu tượng và phương thức thường.	Chỉ chứa phương thức trừu tượng (từ Java 8 có thêm default method).

Thuộc tính	Có thể chứa các biến instance (non-static, non-final).	Chỉ có thể chứa hằng số (public static final).
Constructor	Có Constructor (để lớp con gọi khởi tạo).	Không có Constructor.
Tốc độ	Nhanh hơn Interface một chút.	Chậm hơn do cần tìm phương thức thực thi tương ứng.

5. Khi nào sử dụng loại nào?

Sử dụng Abstract Class khi:

- Bạn muốn chia sẻ code (phương thức thường, thuộc tính) giữa các lớp có quan hệ gần gũi.
- Các lớp con có nhiều điểm chung về bản chất (Quan hệ "IS-A"). Ví dụ: *Cho là một Động Vật*.
- Bạn muốn định nghĩa các biến không phải là hằng số và cho phép lớp con thay đổi trạng thái của chúng.

Sử dụng Interface khi:

- Bạn muốn định nghĩa một khả năng hoặc hành vi cho các lớp không liên quan đến nhau (Quan hệ "HAS-A" hoặc "CAN-DO"). Ví dụ: Lớp *MayBay* và lớp *Chim* đều có thể *Bay* (Interface *Bay*), dù chúng không cùng họ hàng.
- Bạn muốn tận dụng tính đa kế thừa.
- Bạn muốn tạo ra một bộ khung chuẩn (API contract) mà không quan tâm ai sẽ thực thi nó và thực thi như thế nào.

6. Kết luận

Tính trừu tượng là chìa khóa để thiết kế các hệ thống phần mềm lớn.

- **Abstract class** giúp xây dựng bộ khung phân cấp, tái sử dụng mã nguồn.
- **Interface** giúp chuẩn hóa giao tiếp giữa các đối tượng, tạo ra sự linh hoạt và lỏng lẻo (loose coupling) cho hệ thống.

Việc phối hợp nhịp nhàng giữa hai công cụ này sẽ giúp mã nguồn Java trở nên chuyên nghiệp, dễ đọc và dễ bảo trì.